

Snowflake Data Loading and Snowpipe Assignment

Objective -

The objective of this assignment is to:

- Create sample data files in CSV, JSON, and Parquet formats.
 - Create a target table in Snowflake.
 - Upload and stage files in Snowflake.
 - Load data using the COPY INTO command.
 - Configure and test Snowpipe for automated data ingestion.
 - Perform data loading transformations and error handling.
-

Part 1: Prepare Sample Files

Sample CSV File

File Name: employees.csv

```
ID,Name,Age,Salary
1,John,30,50000
2,Alice,25,60000
```

Description

Column	Data Type	Description
ID	Integer	Employee ID
Name	String	Employee Name
Age	Integer	Employee Age
Salary	Float	Employee Salary

Sample JSON File

File Name: employees.json

```
[
  {
    "ID": 1,
    "Name": "John",
    "Age": 30,
    "Salary": 50000
  },
  {
    "ID": 2,
    "Name": "Alice",
    "Age": 25,
    "Salary": 60000
  }
]
```

Description

The JSON file stores employee records as an array of objects.

Sample Parquet File

Using Python and Pandas

```
import pandas as pd
```

```
data = {
    "ID": [1, 2],
    "Name": ["John", "Alice"],
    "Age": [30, 25],
    "Salary": [50000, 60000]
}
```

```
df = pd.DataFrame(data)
```

```
df.to_parquet("employees.parquet")
```

Output

A Parquet file named **employees.parquet** is generated for loading into Snowflake.

Part 2: Create Target Table

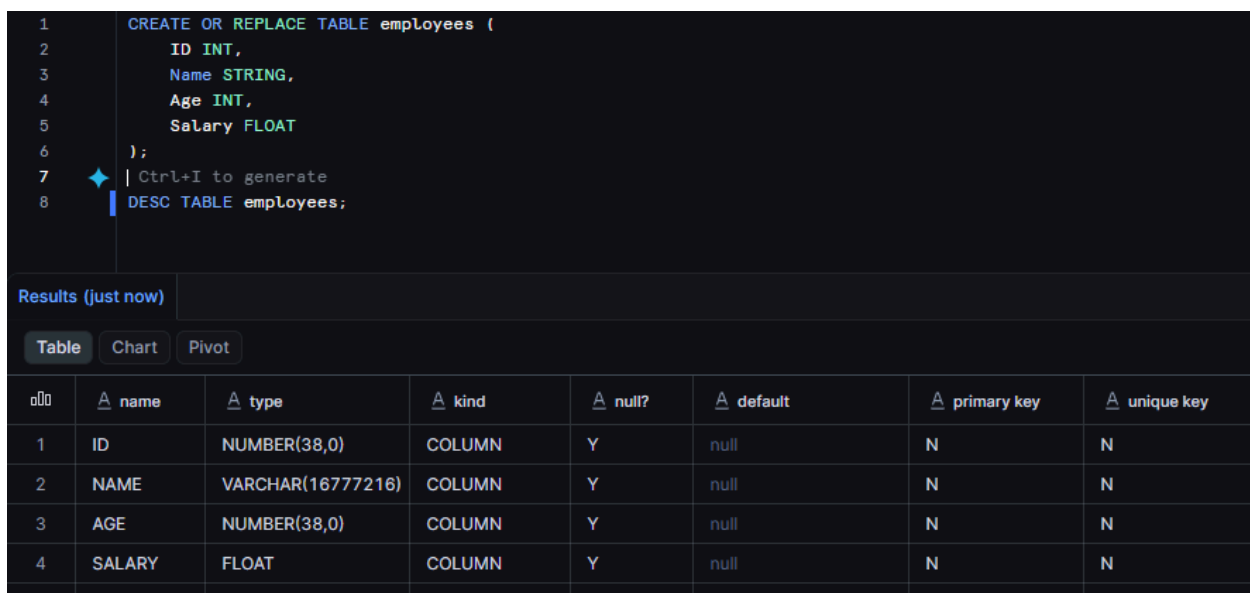
SQL Statement

```
CREATE OR REPLACE TABLE employees (  
  ID INT,  
  Name STRING,  
  Age INT,  
  Salary FLOAT  
);
```

Verification

```
DESC TABLE employees;
```

Expected Result



The screenshot shows a SQL IDE with a dark theme. The top pane contains the SQL statement to create a table named 'employees' with columns ID, Name, Age, and Salary. The bottom pane shows the results of the 'DESC TABLE employees;' command, displaying a table with 8 columns: name, type, kind, null?, default, primary key, and unique key. The table has 4 rows of data corresponding to the columns defined in the CREATE statement.

```
1 CREATE OR REPLACE TABLE employees (  
2   ID INT,  
3   Name STRING,  
4   Age INT,  
5   Salary FLOAT  
6 );  
7 | Ctrl+I to generate  
8 DESC TABLE employees;
```

Results (just now)

Table Chart Pivot

	name	type	kind	null?	default	primary key	unique key
1	ID	NUMBER(38,0)	COLUMN	Y	null	N	N
2	NAME	VARCHAR(16777216)	COLUMN	Y	null	N	N
3	AGE	NUMBER(38,0)	COLUMN	Y	null	N	N
4	SALARY	FLOAT	COLUMN	Y	null	N	N

Part 3: Create an Internal Stage

Create Stage

```
CREATE OR REPLACE STAGE my_stage;
```

Verify Stage

```
SHOW STAGES;
```

Expected Result

The stage **MY_STAGE** should be available for file uploads.

Part 4: Upload Files to Stage

Method 1: Using Snowflake Web UI

Steps -

1. Navigate to **Data » Databases**.
2. Select the target database and schema.
3. Open **Stages**.
4. Select **MY_STAGE**.
5. Click **Upload Files**.
6. Upload:
 - employees.csv
 - employees.json
 - Employees.parquet

Upload Your Files



Drag and drop files here

or

Browse

File size limit: 250MB

Select database, schema and stage

Schema **DEMO_DB.PUBLIC** ▾

Select Stage ▾

 MY_STAGE

Specify the path to an existing folder

Example / Path / To / Folder

Once the files finish uploading, your Directory Table will automatically refresh.

Close

Upload

Method 2: Using SnowSQL

Upload CSV File

```
snowsql -q "PUT file://path/to/employees.csv @my_stage;"
```

Upload JSON File

```
snowsql -q "PUT file://path/to/employees.json @my_stage;"
```

Upload Parquet File

```
snowsql -q "PUT file://path/to/employees.parquet @my_stage;"
```

Verify Uploaded Files

LIST @my_stage;

Expected Result

```
C:\Users\parthk.shah>snowsql -a HFIPXNH-ELC12169 -u PARTH -d DEMO_DB -s PUBLIC -q "PUT file://C:/Users/parthk.shah/Downloads/employee.json @my_stage;"
Password:
* SnowSQL * v1.5.0
Type SQL statements or !help
```

source	target	source_size	target_size	source_compression	target_compression	status	message
employee.json	employee.json.gz	172	128	NONE	GZIP	UPLOADED	

```
1 Row(s) produced. Time Elapsed: 1.851s
Goodbye!
```

Part 5: Load Data Using COPY INTO

Create CSV File Format

```
CREATE OR REPLACE FILE FORMAT csv_format
TYPE = CSV
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = '';
```

Load CSV Data

```
COPY INTO employees
FROM @my_stage/employees.csv
FILE_FORMAT = csv_format;
```

Verify Data

```
SELECT * FROM employees;
```

Expected Output

```
1 CREATE OR REPLACE FILE FORMAT csv_format
2 TYPE = CSV
3 SKIP_HEADER = 1
4 FIELD_OPTIONALLY_ENCLOSED_BY = '';
5
6 COPY INTO employees
7 FROM @MY_STAGE/sample.csv
8 FILE_FORMAT = csv_format;
9
10 SELECT * FROM employees;
```

results (just now)

Table Chart Pivot

2 rows 185ms

#	ID	NAME	AGE	SALARY
1	1	John	30	50000
2	2	Alice	25	60000

Part 6: Create and Configure Snowpipe

Objective

Snowpipe enables continuous and automated data ingestion from staged files.

Create Snowpipe

```
CREATE OR REPLACE PIPE my_pipe AS
COPY INTO employees
FROM @my_stage
FILE_FORMAT = (
  TYPE = 'CSV'
  FIELD_OPTIONALLY_ENCLOSED_BY = ''
  SKIP_HEADER = 1
);
```

Verify Pipe

```
SHOW PIPES;
```

Expected Result

my workspace snowpipe.sql

ACCOUNTADMIN DEMO_WH (X-Small) DEMO_DB PUBLIC Share

```

29
30 DESC INTEGRATION S3_INT;
31
32 CREATE OR REPLACE STORAGE INTEGRATION S3_INT
33 TYPE = EXTERNAL_STAGE
34 STORAGE_PROVIDER = S3
35 ENABLED = TRUE
36 STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::700903221693:role/newrole'
37 STORAGE_ALLOWED_LOCATIONS = ('s3://demo-snowpipe-ps/netflix_titles.csv');
38
39 CREATE OR REPLACE FILE FORMAT CSV_FORMAT
40 TYPE = CSV
41 SKIP_HEADER = 1
42 FIELD_OPTIONALLY_ENCLOSED_BY = '';
43
44 CREATE OR REPLACE STAGE S3_STAGE
45 URL = 's3://demo-snowpipe-ps/netflix_titles.csv'
46 STORAGE_INTEGRATION = S3_INT
47 FILE_FORMAT = CSV_FORMAT;
48
49 LIST @S3_STAGE;
50

```

Results (just now)

Table Chart Pivot 7,787 rows 1.3s

	SHOW_ID	TYPE	TITLE	DIRECTOR	CAST	COUNTRY	DATE_ADDED	RELEASE_YEAR
1	s1	TV Show	3%	null	João Miguel, Bianca Comparato, Michel Gomes, Rodolfo Val	Brazil	August 14, 2020	202
2	s2	Movie	7:19	Jorge Michel Grau	Demián Bichir, Héctor Bonilla, Oscar Serrano, Azalia Ortiz, C	Mexico	December 23, 2016	201
3	s3	Movie	23:59	Gilbert Chan	Tedd Chan, Stella Chung, Henley Hill, Lawrence Koh, Tommy	Singapore	December 20, 2018	201
4	s4	Movie	9	Shane Acker	Elijah Wood, John C. Reilly, Jennifer Connelly, Christopher P	United States	November 16, 2017	200
5	s5	Movie	21	Robert Luketic	Jim Sturgess, Kevin Spacey, Kate Bosworth, Aaron Yoo, Lizi	United States	January 1, 2020	200
6	s6	TV Show	46	Serdar Akar	Erdal Beşikçioğlu, Yasemin Allen, Melis Birkan, Saygin Soysa	Turkey	July 1, 2017	201
7	s7	Movie	122	Yasir Al Yasiri	Amina Khalil, Ahmed Dawood, Tarek Lotfy, Ahmed El Fishaw	Egypt	June 1, 2020	201
8	s8	Movie	187	Kevin Reynolds	Samuel L. Jackson, John Heard, Kelly Rowan, Clifton Collins	United States	November 1, 2019	199
9	s9	Movie	706	Shravan Kumar	Divya Dutta, Atul Kulkarni, Mohan Aqashe, Anupam Shyam, I	India	April 1, 2019	201

Part 7: Trigger Snowpipe

Manual Refresh

ALTER PIPE my_pipe REFRESH;

Purpose

This command scans staged files and loads any new files into the target table.

Verify Load Status

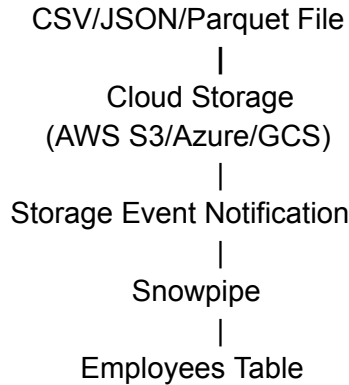
SELECT * FROM employees;

Expected Result

Data from newly staged files should appear in the table.

Part 8: Automated Snowpipe Processing

Architecture



Workflow

1. File uploaded to cloud storage.
2. Event notification generated.
3. Snowpipe receives notification.
4. Snowpipe executes COPY INTO automatically.
5. Data loaded into Snowflake table.

Part 9: COPY INTO with Error Handling

Basic COPY Command

```
COPY INTO employees
FROM @my_stage/employees.csv
FILE_FORMAT = (
  TYPE = 'CSV'
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  SKIP_HEADER = 1
);
```

COPY Command with Error Handling

```
COPY INTO employees
FROM @my_stage/employees.csv
FILE_FORMAT = (
  TYPE = 'CSV'
```

```
    SKIP_HEADER = 1
)
ON_ERROR = 'CONTINUE'
FORCE = TRUE;
```

Parameters

Parameter	Purpose
ON_ERROR='CONTINUE'	Skips problematic records
FORCE=TRUE	Reloads files even if previously loaded

Part 10: Validation and Monitoring

Check Load History

```
SELECT *
FROM TABLE(
    INFORMATION_SCHEMA.COPY_HISTORY(
        TABLE_NAME=>'EMPLOYEES',
        START_TIME=>DATEADD('DAY',-1,CURRENT_TIMESTAMP()) )
);
```

View Pipe Status

```
SELECT SYSTEM$PIPE_STATUS('MY_PIPE');
```

View Staged Files

```
LIST @my_stage;
```

Conclusion

This assignment demonstrated Snowflake's data ingestion capabilities using stages, COPY INTO commands, and Snowpipe. Data was successfully loaded from CSV, JSON, and Parquet files into a Snowflake table. Snowpipe provided automated ingestion functionality, while COPY INTO options enabled flexible and reliable data loading with error handling and transformation support.